Charting & Visualization

> Part of the Streamlit curriculum. This document covers Plotly Express and Graph Objects in depth, plus other charting options worth knowing about.

1. Streamlit's own built-in chart functions

For very quick, simple charts, Streamlit has built-in wrappers requiring no extra library:

```
st.line_chart(df, x="date", y="value")
st.bar_chart(df, x="category", y="value")
st.area_chart(df, x="date", y="value")
```

These are convenient for a first pass but limited — no titles, no custom legends, no hover tooltips beyond the defaults, and no chart types beyond the handful provided. For anything resembling a real, presentable dashboard, reach for a full charting library instead — **Plotly** is a common pairing with Streamlit given its native interactivity (zoom, pan, hover tooltips) and the two-tier API described below.

2. Plotly Express: quick, declarative charts

`plotly.express` (commonly imported as `px`) builds a complete, styled chart from a tidy DataFrame in one function call:

```
import plotly.express as px

fig = px.line(df, x="date", y="value", color="category", title="Value over time")
st.plotly_chart(fig, use_container_width=True)

fig2 = px.bar(df, x="category", y="value", title="Value by category")
st.plotly_chart(fig2, use_container_width=True)

fig3 = px.bar(
    df, x="category", y="value", color="value",
    color_continuous_scale=["green", "lightgrey", "crimson"],   # a diverging scale, useful for +/- values
    title="Value by category, colored by magnitude",
)
```

Common Plotly Express chart functions: `px.line` (trends over a continuous axis, usually time), `px.bar` (categorical comparison), `px.scatter` (relationship between two continuous variables), `px.area` (stacked/cumulative trends). Each accepts a `color=` argument to split the chart into multiple series/colors by a

categorical column, and a `labels={}` dict to override axis/legend text without renaming the underlying DataFrame columns.

3. Plotly Graph Objects: custom and composite charts

The moment you need something Express has no direct shape for — most commonly a shaded confidence-interval/uncertainty band, or several different trace types layered on one chart — drop down to `plotly.graph_objects` (commonly imported as `go`) and build the figure trace-by-trace:

```
import plotly.graph_objects as go
import pandas as pd

fig = go.Figure()

fig.add_trace(go.Scatter(
    x=x, y=y_mean, name="Forecast", mode="lines", line=dict(color="green", width=2),
))

# Shaded confidence-interval band: concatenate x with its own reverse, and the
# upper CI bound with the reversed lower CI bound, then fill the enclosed shape.
fig.add_trace(go.Scatter(
    x=pd.concat([x, x[::-1]]),
    y=pd.concat([y_upper, y_lower[::-1]]),
    fill="toself",
    fillcolor="rgba(0,128,0,0.15)",
    line=dict(color="rgba(255,255,255,0)"),   # invisible border — only the fill shows
    name="95% Confidence Interval",
))

fig.add_trace(go.Bar(x=categories, y=values, name="Comparison bars"))   # mixing trace types on one figure

fig.update_layout(
    title="Forecast with confidence interval",
    xaxis_title="Date", yaxis_title="Value",
    legend=dict(orientation="h", yanchor="bottom", y=1.02, xanchor="right", x=1),
)
st.plotly_chart(fig, use_container_width=True)
```

The confidence-band pattern above (concatenate the x-values with their own reverse, and the upper CI bound with the reversed lower CI bound, then `fill="toself"`) is the standard idiom for drawing a single closed shaded region between two bounding lines — worth memorizing since it comes up constantly whenever a forecast or estimate needs its uncertainty visualized.

3.1 When to reach for Graph Objects instead of Express

Express is built for a fixed set of common chart shapes expressed in one call from a tidy DataFrame. Reach for Graph Objects specifically when you need:

- A confidence/uncertainty band (as above).
- Multiple different trace *types* on one figure (e.g., bars and a line together, as in the final example above).
- Fine control over a specific trace's appearance that Express doesn't expose a parameter for.

For everything else, Express's one-line calls are faster to write and read.

4. Accessibility

Interactive charts rendered as SVG/canvas elements are often invisible to screen readers by default. Where the charting library or your own wrapper supports it, provide an explicit textual description of what the chart shows (its title, the range of values, the key takeaway) so the chart is meaningfully described to assistive technology, not just visually presentable.

5. Other charting options worth knowing about

Library	Notable trait
Altair	A declarative "grammar of graphics" API (similar in spirit to R's ggplot2); very concise for statistical/exploratory chart types
Matplotlib	The most widely-used Python plotting library overall; static (non-interactive) by default, but the most flexible for fully custom, publication-quality figures
Bokeh	Another interactive-charting option, with a different API design philosophy than Plotly

Any of these can be rendered inside a Streamlit app with the corresponding `st.altair_chart` / `st.pyplot` / `st.bokeh_chart` function. Plotly's popularity in Python-data-science-adjacent dashboards specifically comes from its interactivity (zoom/pan/hover) combined with a reasonably approachable API — but the "right" charting library ultimately depends on whether interactivity, static publication quality, or a particular declarative style matters most for your use case.

6. Core libraries summary

Library	Used for
Plotly Express	quick, declarative charts from a tidy DataFrame
Plotly Graph Objects	custom/composite charts (confidence bands, multi-trace overlays)
Altair / Matplotlib / Bokeh	alternative charting libraries, each with different trade-offs

See also

- Streamlit index
- Core Concepts — the widgets and layout these charts are typically embedded within